Great Place To Work. Certified FEB 2023 – FEB 2024 INDIA

UPES

UNIVERSITY OF TOMORROW

nirf RANKING 2025

*RANKED 45 IN INDIA

#University Category

QS WORLD UNIVERSITY RANKINGS

NO.1 PVT. UNIVERSITY IN ACADEMIC REPUTATION IN INDIA

NAAC

ACCREDITED **GRADE 'A'** BY NAAC

PERFECT SCORE OF **150/150** AS A TESTAMENT TO EXECEPTIONAL E-LEARNING METHODS

UPES UNIVERSITY OF TOMORROW

Applied Machine Learning

Unit 06 – Lecture 18: Support Vector Machines

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

Of all the lines that separate the data, one is furthest from it

Topic focus: **the margin, and the handful of points that set it**

Repository: `https://github.com/tali7c/Applied-Machine-Learning`

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 8–9.;James et al., *An Introduction to Statistical Learning with Python*, Ch. 8–9.;Bishop, *Pattern Recognition and Machine Learning*, Ch. 4–5, 7, 14.;Zhang et al., *Dive into Deep Learning*, Ch. 5

Quick Links

The maximum margin | Primal, Lagrangian, dual | Support vectors | Soft margin and C

The hinge loss | The kernel trick | γ, scaling, cost | Summary

Agenda

The maximum-margin idea

The primal, the Lagrangian and the dual

Support vectors: the memorable result

The soft margin and C

The loss the SVM minimises

The kernel trick

γ, scaling and what it all costs

Summary

How this lecture works

Every idea appears twice

1. **By hand** — six points on graph paper, two vectors with small integer entries, one multiplier per support vector. Small enough that every number can be checked with a pen.
2. **In code** — the same arithmetic, then the same arithmetic again inside `scikit-learn`, so you can see that `SVC` returns *exactly* the coefficients you worked out and nothing magic.

Commit before you look

Four slides ask you to predict a number before it is revealed. Write your guess down. Being wrong on paper is how the idea sticks.

The centrepiece is **six points solved completely by hand**: $w = (1, 1)$, $b = -3$, two support vectors, margin width $\sqrt{2}$, and both multipliers equal to 1. Then we delete one point at a time and watch what does and does not move.

Where we are

Two lectures ago the objection was instability

A single tree moves when you move a few rows. Ensembles answered that by averaging many unstable models.

This lecture answers it a different way

Build *one* boundary, and choose the one that is as far as possible from every training point. A boundary with room around it does not move when the data jiggles — because most of the data was ne

Today's one question

A hundred different lines separate these two classes perfectly. *Which one should I pick?*

And a promise for the second half

We will take a dataset that **no line can separate**, and classify it perfectly with a linear model — without ever computing the coordinates of the space in which it becomes linear. That is the kernel trick, and we will verify the identity that makes it work rather than assert it.

One seed — and the one thing no seed can fix

The one seed

Everything arbitrary in this deck uses **seed** 0: the cross-validation shuffle (`StratifiedKFold(5, shuffle=True, random_state=0)`), the train/test split, the solver permutations. Every listing shows it.

A fit with no seed at all

Given its data, an `SVC` fit is *deterministic*. Every accuracy, every support-vector count, every dual coefficient in this deck reproduces exactly.

Seeds that define data

`make_blobs(random_state=3)`, `make_moons(random_state=3)`, `make_circles(random_state=0)`, `make_classification(random_state=0)`. These do not randomise an experiment — they *are* the dataset. They never move.

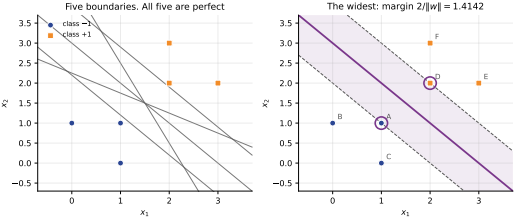
And the exception

The cost section reports **seconds**. No seed fixes a clock. Quote the support-vector counts beside them instead.

Six points, and more separating lines than you can draw

The data

point	x	y
A	$(1, 1)$	-1
B	$(0, 1)$	-1
C	$(1, 0)$	-1
D	$(2, 2)$	$+1$
E	$(3, 2)$	$+1$
F	$(2, 3)$	$+1$



Every line in the picture gets all six right.
Training accuracy cannot tell them apart.

Predict first #1 — which line would you bet on?

Commit to an answer

Five candidate lines, all with zero training errors. Rank them by how *safe* they look, and then put a number on it: for the line $x_1 + x_2 - 3 = 0$, how far is the nearest of the six points?

Distance = ? and how does it compare with $x_1 + x_2 - 3.9 = 0$?

Predict first #1 — which line would you bet on?

Commit to an answer

Five candidate lines, all with zero training errors. Rank them by how *safe* they look, and then put a number on it: for the line $x_1 + x_2 - 3 = 0$, how far is the nearest of the six points?

Distance = ? and how does it compare with $x_1 + x_2 - 3.9 = 0$?

$x_1 + x_2 - 3 = 0$ has its nearest point at $\mathbf{0.7071}$; $x_1 + x_2 - 3.9 = 0$ at $\mathbf{0.0707}$ — ten times closer, same accuracy.

The same five lines, measured

```
  line                    errors   nearest point   margin width
  1x1 +1x2 -2.2 = 0            0          0.1414         0.2828
  1x1 +1x2 -3.9 = 0            0          0.0707         0.1414
  2x1 +1x2 -4.5 = 0            0          0.6708         1.3416
  1x1 +2x2 -4.5 = 0            0          0.6708         1.3416
  1x1 +1x2 -3 = 0              0          0.7071         1.4142
every one of them is a perfect classifier; they are not equally safe
```

Accuracy is blind to this entirely. The last line has ten times the clearance of the second and scores identically on the training set. The whole of today is a way of making that difference into an objective a solver can optimise.

Distance from a point to a hyperplane

A hyperplane is $\{x : w^\top x + b = 0\}$. The vector w is perpendicular to it: if x and x' both lie on it, $w^\top(x - x') = 0$.

The signed distance

Write $x = x_p + r\frac{w}{\|w\|}$, with x_p the foot of the perpendicular. Then

$$w^\top x + b = \underbrace{w^\top x_p + b}_{=0} + r\frac{w^\top w}{\|w\|} = r\|w\|,$$

so the distance is $r = \dfrac{w^\top x + b}{\|w\|}$.

Two different margins

Functional margin: $\hat{\gamma}_i = y_i(w^\top x_i + b)$.

Geometric margin: $\gamma_i = \hat{\gamma}_i / \|w\|$.

Doubling w and b doubles the functional margin and leaves the geometric one alone — and leaves the boundary exactly where it was.

Deriving the width: the margin is $2/\|w\|$

Kill the redundancy first

(w, b) and (cw, cb) describe the *same* boundary for any $c > 0$. Fix the scale by demanding that the closest points have functional margin exactly one:

$$\min_i \; y_i \left(w^\top x_i + b\right) = 1. \qquad \text{(the } \textit{canonical} \text{ hyperplane)}$$

The consequence

A closest point of class $+1$ satisfies $w^\top x + b = +1$; one of class -1 satisfies $w^\top x + b = -1$. Their distances from the boundary are $1/\|w\|$ each, so the empty band between them has width

And therefore

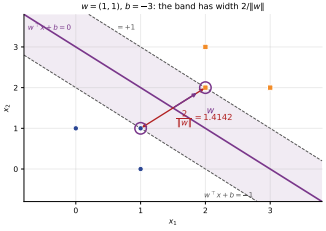
$$\max \frac{2}{\|w\|} \iff \min \|w\| \iff \min \tfrac{1}{2}\|w\|^2.$$

make the derivative tidy; they change

The whole problem, in one box — and drawn

The hard-margin primal

$$\min_{w,\,b} \tfrac{1}{2}\|w\|^2 \qquad \text{subject to} \qquad y_i\,(w^\top x_i + b) \;\geq\; 1 \quad \text{for } i = 1, \ldots, n.$$



$w = (1, 1)$, $b = -3$, $\|w\| = \sqrt{2}$, so the band is $2/\|w\| = \mathbf{1.414214}$ wide; **A** and **D** sit exactly on its edges. The constraint says two things at once — correct side *and* at least one functional unit clear. This is a **convex quadratic program**: one global optimum, no local minima to get stuck in. And note what is *absent* from the objective — any count of how many points are correct.

Attaching a multiplier to every constraint

The Lagrangian

Introduce one multiplier $\alpha_i \geq 0$ per training point:

$$L(w, b, \alpha) \;=\; \tfrac{1}{2}\|w\|^2 \;-\; \sum_{i=1}^{n} \alpha_i \big[\, y_i(w^\top x_i + b) - 1 \,\big].$$

Stationarity

$$\frac{\partial L}{\partial w} = 0 \implies w = \sum_i \alpha_i y_i x_i$$

$$\frac{\partial L}{\partial b} = 0 \implies \sum_i \alpha_i y_i = 0$$

Read the first one again

w is a **weighted sum of the training points themselves**. The model does not invent a direction; it combines the data. Any point with $\alpha_i = 0$ contributes

Substituting back gives the dual

Put $w = \sum_i \alpha_i y_i x_i$ and $\sum_i \alpha_i y_i = 0$ back into L. The b term vanishes, the cross terms collapse, and what is left contains the data *only* through inner products:

The dual problem

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j \, y_i y_j \, \langle x_i, x_j \rangle \quad \text{s.t.} \quad \alpha_i \geq 0, \quad \sum_i \alpha_i y_i = 0.$$

Why anyone bothers

The problem is now in n variables instead of $d + 1$, which is a win when $d \gg n$ — and, far more importantly, x appears **only** inside $\langle x_i, x_j \rangle$.

Hold on to that

Everything in the kernel section follows from this one observation. If the algorithm only ever needs inner products, you can change what "inner product"

The KKT conditions, and the sentence that matters

At the optimum of a convex problem with linear constraints

$$\alpha_i \geq 0, \qquad y_i(w^\top x_i + b) - 1 \geq 0, \qquad \underbrace{\alpha_i \big[\, y_i(w^\top x_i + b) - 1 \,\big] = 0}_{\textbf{complementary slackness}}.$$

A product of two things is zero

So for every single point, *at least one factor is zero*:

- either $\alpha_i = 0$ — the point is irrelevant;
- or $y_i(w^\top x_i + b) = 1$ — the point sits exactly on the margin.

The definition, earned rather than announced

The points with $\alpha_i > 0$ are the **support vectors**.

Everything else could be deleted and the answer would not change. We are about

By hand: solving the six points completely

Guess the support set, then verify

The two closest points across the classes are A$(1,1)$ and D$(2,2)$. Suppose only those two have $\alpha > 0$. Then $\sum_i \alpha_i y_i = 0$ gives $-\alpha_A + \alpha_D = 0$, so $\alpha_A = \alpha_D = \alpha$, and

$$w = \alpha\big[(2,2) - (1,1)\big] = \alpha\,(1,1).$$

Two equations, two unknowns

Both are on the margin, so

$$-(2\alpha + b) = 1, \qquad 4\alpha + b = 1.$$

Add them: $2\alpha = 2$, so $\boxed{\alpha = 1}$ and $b = 1 - 4 = \boxed{-3}$, giving $w = \boxed{(1,1)}$.

Check the other four: B and C give $w^\top x + b = -2$, so $y_i f = 2 > 1$; E and F give $+2$, so $y_i f = 2 > 1$. Every constraint holds, so the guess was right. Width $2/\|w\| = 2/\sqrt{2} = 1.414214$.

The same thing in code — to ten decimal places

```
X = np.array([[1.,1.],[0.,1.],[1.,0.],[2.,2.],[3.,2.],[2.,3.]])
y = np.array([-1,-1,-1, 1, 1, 1])
clf = SVC(kernel="linear", C=1e6).fit(X, y) # C huge = hard margin
```

```
by hand :  w = (1, 1)   b = -3
           ||w||   = 1.4142135624
           2/||w|| = 1.4142135624
SVC     :  w = [1. 1.]   b = -3.0000000000
           2/||w|| = 1.4142135624
agree to 1e-9: True
```

Not "close to". **The same numbers.** The library solved the same quadratic program you just solved with two linear equations.

The multipliers, and w rebuilt from them

```
alpha = np.zeros(len(X)); alpha[clf.support_] = np.abs(clf.dual_coef_[0])
w_dual = (alpha * y) @ X
```

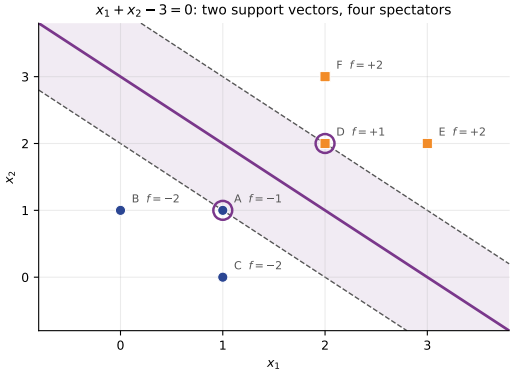
```
support_         : [0 3] -> A D
alpha            : [1. 0. 0. 1. 0. 0.]
sum_i alpha_i y_i = 0.0e+00   (must be 0)
w = sum_i alpha_i y_i x_i = [1. 1.]
b = mean over SVs of (y_s - w.x_s) = -3.0000000000
dual objective   = 1.0000000000
(1/2)||w||^2     = 1.0000000000
```

Four of the six multipliers are **exactly zero**. The dual objective equals the primal objective, which is what *strong duality* means: solving either one solves the other.

Which points are on the margin? Read $y_i f(x_i)$

pt	f(x)	y f(x)	on the margin?
A	-1.00	+1.00	yes
B	-2.00	+2.00	no
C	-2.00	+2.00	no
D	+1.00	+1.00	yes
E	+2.00	+2.00	no
F	+2.00	+2.00	no

$y_i f(x_i) = 1$ exactly on the margin, > 1 strictly outside. Compare against the α column: complementary slackness, printed twice.



Two rings, two non-zero multipliers, both equal to 1.
One line, fixed by two points — the smallest number

Predict first #2 — delete a point

Commit to two answers

I am going to refit the SVM twice on five points instead of six.

1. First I delete **B** at $(0, 1)$, which is not a support vector. What happens to w, to b and to the margin width?
2. Then I put B back and delete **A** at $(1, 1)$, which is. Same three questions.

Write down both answers before the next slide.

Predict first #2 — delete a point

Commit to two answers

I am going to refit the SVM twice on five points instead of six.

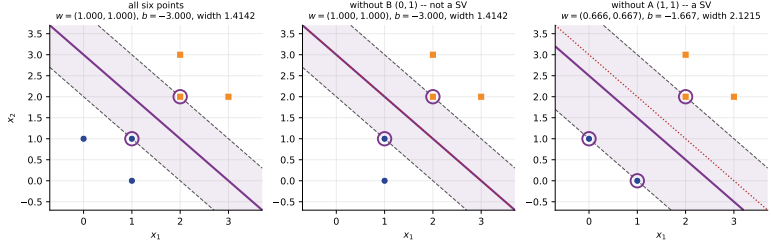
1. First I delete **B** at $(0, 1)$, which is not a support vector. What happens to w, to b and to the margin width?
2. Then I put B back and delete **A** at $(1, 1)$, which is. Same three questions.

Write down both answers before the next slide.

Almost everyone gets the first one right in words ("nothing happens") and hedges. It is worth being precise: *nothing happens* means the difference is $0.00\mathrm{e}{+00}$, not "a small change".

The reveal — one deletion is free, the other is not

```
all six points              w = (1.0000, 1.0000)  b = -3.0000  nSV = 2  2/||w|| = 1.4142
drop B (0,1), not a SV      w = (1.0000, 1.0000)  b = -3.0000  nSV = 2  2/||w|| = 1.4142
drop D (2,2), a SV          w = (0.6665, 0.6670)  b = -2.3337  nSV = 3  2/||w|| = 2.1211
drop A (1,1), a SV          w = (0.6664, 0.6668)  b = -1.6665  nSV = 3  2/||w|| = 2.1215
dropping B moved w by 0.00e+00 and b by 0.00e+00 -- nothing moved
dropping a support vector widened the margin 1.4142 -> 2.1211
```



Why the margin got wider

Constraints only ever hurt

Deleting a point removes a constraint from the primal. A minimisation over a *larger* feasible set can only do the same or better, so $\frac{1}{2}\|w\|^2$ can only fall — and a smaller $\|w\|$ is a wider band.

Check it by hand

Without A, the nearest negative to the positives is the midpoint of B$(0,1)$ and C$(1,0)$, at $(0.5, 0.5)$; the nearest positive is D$(2,2)$. The gap is $\|(1.5, 1.5)\| = 2.1213$, and $2/\|w\| = 2.1215$. Agreement to four dec

The examinable sentence

The support vectors are the training set.

Delete every other row and you get the identical model. Delete one support vector and the boundary moves. That is what $\alpha_i = 0$ *means*.

And a warning

The same property makes an SVM sensitive to an outlier that happens to sit near the boundary — it becomes a

model. The soft margin exists to plum

Not a toy effect: throw away 69 of 80 rows

```
full = SVC(kernel="linear", C=1.0).fit(Xb, yb) # 80 points
only = SVC(kernel="linear", C=1.0).fit(Xb[full.support_], yb[full.support_
    ])
```

```
trained on all 80 points : w = [-0.976353 -0.408819]  b = -1.475724
trained on the 11 SVs only: w = [-0.975901 -0.40859 ]  b = -1.475336
identical predictions on all 80 points: True
the other 69 rows changed nothing
```

86% of the training set could be deleted after the fact with no change to a single prediction. The coefficients differ in the fourth decimal only because the solver stops at its tolerance, not because the answer is different.

Real data is not separable, and hard margin then fails

The hard-margin problem is infeasible

If no hyperplane separates the classes, there is no (w, b) satisfying every $y_i(w^\top x_i + b) \geq 1$. The feasible set is empty. The solver does not return a poor answer; there is no answer.

Buy your way out with slack

Give each point a **slack variable** $\xi_i \geq 0$ and let it fall short:

$$y_i(w^\top x_i + b) \;\geq\; 1 - \xi_i.$$

Then charge for it in the objective.

The soft-margin primal

$$\min_{w,b,\xi} \;\; \tfrac{1}{2}\|w\|^2 \;+\; C\sum_{i=1}^{n} \xi_i \qquad \text{s.t.} \qquad y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

What ξ_i measures, in three cases

$\xi_i = 0$

The point is outside the margin, on the right side. It has paid nothing. Most points are here.

$0 < \xi_i \leq 1$

Inside the margin band but still correctly classified. It costs something even though the prediction is right.

$\xi_i > 1$

Past the boundary — **misclassified**. The count of these is the training error.

$\xi_i = \max\big(0,\ 1 - y_i f(x_i)\big)$ — and you have met that function before. Hold that thought for two slides.

The dual is *unchanged* except that $\alpha_i \geq 0$ becomes $0 \leq \alpha_i \leq C$. That single box constraint is the entire mathematical content of the soft margin.

C trades margin width against violations

C	nSV	2/‖w‖	sum of slacks	inside margin	wrong side	train acc
0.01	35	5.2135	17.8304	32	3	0.9625
0.1	18	2.1750	8.3313	15	3	0.9625
1	11	1.8895	7.6247	8	3	0.9625
10	9	1.5278	7.5279	6	3	0.9625
1000	8	1.3483	7.5128	5	3	0.9625

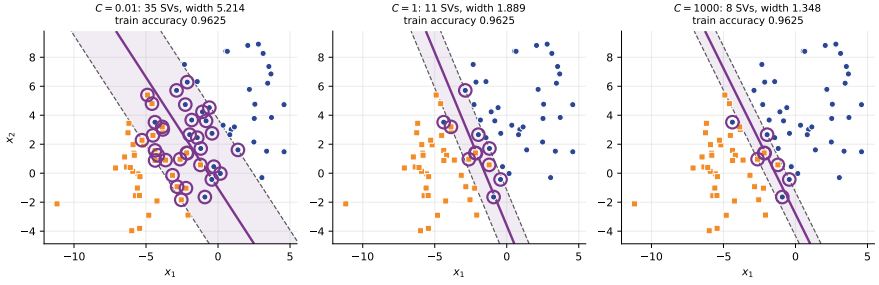
Read the second and fifth columns together

$C = 0.01$ buys a band 5.21 wide by letting 32 points sit inside it. $C = 1000$ squeezes the band to 1.35 and only 5 remain.

Report the honest part too

Training accuracy is 0.9625 **in every row**, and the same three points are misclassified throughout. On this data C changes the geometry a great deal and the training score not at all — which is precisely why you cannot tune it on the training set.

Drawn: the same three fits



Left to right, $C = 0.01$, 1, 1000. The shaded band narrows; the ringed support vectors go from 35 to 8. Small C means a *soft*, tolerant, well-populated margin.

Strong duality, and the three kinds of α

```
primal  (1/2)||w||^2 + C*sum(hinge) = 8.184919
dual    sum(a) - 1/2 sum a_i a_j y_i y_j x_i.x_j = 8.184597
gap                                  = 3.22e-04
0 < alpha < C  (on the margin)  : 3 points
alpha = C      (inside or wrong): 8 points
alpha = 0      (spectators)     : 69 points
```

The gap of 3.22×10^{-4} is the solver's stopping tolerance (`libsvm` defaults to 10^{-3}), not a real duality gap: for a convex QP the gap at the true optimum is zero. **The** 11 **support vectors are the** 3 **on the margin plus the** 8 **at the ceiling.**

Predict first #3 — is a large C more or less regularised?

Commit before you look

`breast_cancer`, scaled, linear kernel, five-fold cross-validation. I sweep C from 0.001 to 100.

Does $\|w\|$ go **up** or **down** as C grows? And which end of the sweep is the *more* regularised model?

Predict first #3 — is a large C more or less regularised?

Commit before you look

`breast_cancer`, scaled, linear kernel, five-fold cross-validation. I sweep C from 0.001 to 100.

Does $\|w\|$ go **up** or **down** as C grows? And which end of the sweep is the *more* regularised model?

$\|w\|$ climbs from **0.3555** to **20.8998**.

Large C is less regularisation. Most of the room gets this backwards.

C is inverse regularisation — and here is why

```
      C      ||w||    5-fold CV   nSV
  0.001     0.3555    0.9385      252
   0.01     0.7265    0.9701      118
    0.1     1.4800    0.9772       60
      1     3.0669    0.9754       40
     10     7.9741    0.9631       37
    100    20.8998    0.9613       31
large C -> large ||w|| -> narrow margin -> LESS regularisation
```

Divide the objective by C

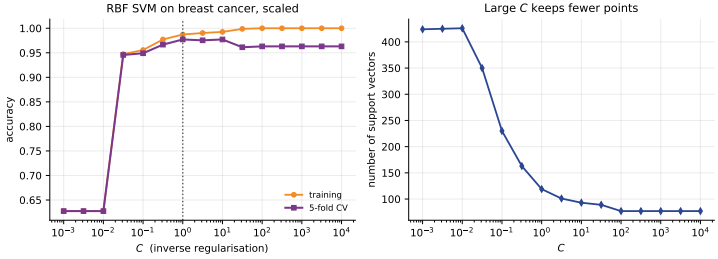
$\frac{1}{2}\|w\|^2 + C\sum \xi_i$ becomes $\frac{1}{2C}\|w\|^2 + \sum \xi_i$. The penalty on $\|w\|^2$ is $1/(2C)$ — so it is $\lambda = 1/C$ that plays the role of ridge's λ.

The mnemonic

Big C = big Cost of error = the model tries hard to fit every point = complex model.

Best CV here is **0.9772** at $C = 0.1$, and both ends are worse. It is a genuine tuning parameter.

The validation curve, drawn



Training accuracy climbs monotonically to 1.0000 and stays there; cross-validated accuracy peaks at **0.9771** near $C = 1$ and then falls to 0.9631. The support-vector count drops from 426 to 77 across the same sweep. **The gap between the two curves is the diagnostic**, not either curve alone.

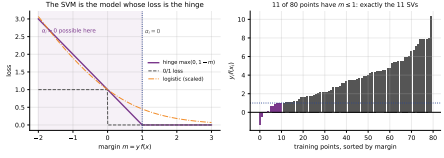
You have already met this function

Eliminate the slack variables

At the optimum, ξ_i is as small as its constraints allow: $\xi_i = \max(0,\ 1 - y_i f(x_i))$. Substitute, and the constrained problem becomes an *unconstrained* one:

$$\min_{w,b} \underbrace{\sum_{i=1}^{n} \max(0,\ 1 - y_i f(x_i))}_{\textbf{hinge loss}} + \frac{1}{2C} \|w\|^2.$$

The hinge loss came up in the loss-functions lecture as a curve on a graph. **The SVM is the model whose loss that is.**



The hinge is **exactly zero** for $m > 1$, so a point comfortably outside the margin contributes no loss *and no gradient*: its multiplier is zero and it is not a support vector. Right panel: **11** of 80 points have $m \le 1$ — and `n_support_` is **11**. The same eleven.

Same loss, two different solvers; and against logistic

```
LinearSVC(loss='hinge', C=0.01)
   ||w|| = 0.7599   mean hinge = 0.104136   objective = 0.8812   acc = 0.9842
SGDClassifier(loss='hinge', matched alpha)
   ||w|| = 0.7389   mean hinge = 0.105196   objective = 0.8716   acc = 0.9719
same loss, same regulariser, two different solvers
```

```
SVC(linear)          uses 40 of 569 rows (the support vectors)
LogisticRegression   uses 569 of 569 rows (every one has a gradient)
cosine between the two weight vectors = 0.9132
5-fold CV  SVC 0.9754   LogisticRegression 0.9789
```

Logistic loss is never exactly zero, so *every* row contributes a gradient forever — no sparsity, but smooth probabilities. The two weight vectors point almost the same way (cosine 0.9132) and score within 0.0035 of each other. **Different loss, similar answer, very different internals.**

A problem no line can solve

Concentric circles

220 points, an inner blob and an outer ring. The class is determined entirely by the *radius*. No straight line in the plane can express "inside a circle".

Measured

A linear SVM scores **0.5682** on its own training data — barely above the majority class. It is not badly tuned; the model class cannot do it.

The idea

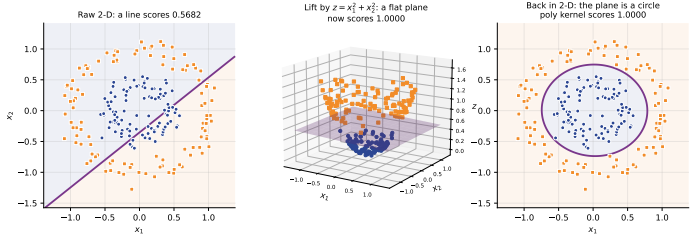
Add a column. Let

$$z = x_1^2 + x_2^2.$$

In (x_1, x_2, z) space the inner blob sits low and the ring sits high, and a **flat plane** $z = c$ separates them.

Coming back down to two dimensions, that plane is a circle. Linear model, curved boundary.

Lift, separate, come back down




```
linear SVM, raw (x1,x2)              accuracy 0.5682
linear SVM, (x1,x2,x1^2+x2^2)        accuracy 1.0000
degree-2 polynomial kernel, raw      accuracy 1.0000
the lifted plane: w = [ 0.2076 -0.0436  6.3956]  b = -3.4436
```

0.5682 → **1.0000** from one extra column. And look at w: the two original coordinates get weights near zero, the radius column gets 6.3956. The model found the right feature by itself.

The trick: never build the new columns

The map, written out

For $x = (x_1, x_2)$, define the degree-two feature map

$$\varphi(x) = \left(1,\ \sqrt{2}\,x_1,\ \sqrt{2}\,x_2,\ x_1^2,\ x_2^2,\ \sqrt{2}\,x_1 x_2\right) \in \mathbb{R}^6.$$

The claim

$$\langle \varphi(x), \varphi(z) \rangle \;=\; \big(1 + \langle x, z \rangle\big)^2.$$

Left side: build two 6-vectors, then multiply. Right side: one dot product in 2-D, add one, square.

Why it matters

The dual needs $\langle x_i, x_j \rangle$ and *nothing else*. Replace every one with $K(x_i, x_j)$ and you have trained a linear SVM in the six-dimensional space — **without ever writing down a six-dimensional vector**.

That claim is an identity. We are going to check it, not believe it.

Verified by hand: $x = (1, 2)$, $z = (3, 1)$

```
x = [1. 2.]   z = [3. 1.]
x.z           = 5.0000000000
(1 + x.z)^2   = 36.0000000000
phi(x) = [1.       1.414214 2.828427 1.       4.       2.828427]
phi(z) = [1.       4.242641 1.414214 9.       1.       4.242641]
phi(x).phi(z) = 36.0000000000
difference    = 0.00e+00
2 features -> 6, and the kernel never built either vector
```

Do the right-hand side with a pen: $x \cdot z = 3 + 2 = 5$, and $(1 + 5)^2 = 36$. Now the left-hand side, term by term: $1 + 6 + 4 + 9 + 4 + 12 = 36$. **The same number, and the difference is exactly zero.**

And over a whole dataset, not one lucky pair

```
V = rng.normal(size=(200, 2))
Kk = (1 + V @ V.T) ** 2 # the kernel, 2-D arithmetic
Phi = np.array([phi2(v) for v in V]) # the explicit 6-D map
Ke = Phi @ Phi.T # its Gram matrix
```

```
kernel matrix       : 200 x 200
explicit map        : 200 x 6, then a 200 x 200 Gram matrix
max |K - phi phi^T| = 2.842e-14
max |K - sklearn|   = 0.000e+00
multiplications, explicit map: 240000
multiplications, kernel      : 80000
```

40 000 entries and the worst disagreement is 2.8×10^{-14} — floating point, not mathematics. Against `polynomial_kernel` it is **0**.

The map you did not have to build

```
d =   2, degree 2 -> explicit map has        6 features
d =  10, degree 2 -> explicit map has       66 features
d =  10, degree 3 -> explicit map has      286 features
d =  30, degree 3 -> explicit map has     5456 features
d =  30, degree 5 -> explicit map has   324632 features
d = 100, degree 3 -> explicit map has   176851 features
d = 100, degree 5 -> explicit map has 96560646 features
the kernel evaluates every one of them with d + 1 multiplications
```

The count is $\binom{d+p}{p}$. At $d = 100$ and degree 5 the explicit map has **ninety-six million** columns, and $(1 + x^\top z)^5$ costs a hundred multiplications, one addition and one power. This is not an optimisation — it is the difference between possible and impossible.

The kernel machine is the explicit-map machine

```
Pk = SVC(kernel="poly", degree=2, gamma=1.0, coef0=1.0, C=1.0).fit(Xr, yr)
Pe = SVC(kernel="linear", C=1.0).fit(np.array([phi2(v) for v in Xr]), yr)
```

```
kernel machine, first five f(x) : [-1.110765  1.463534  2.436074 -1.060217 -1.397726]
explicit map,   first five f(x) : [-1.110765  1.463534  2.436074 -1.060217 -1.397726]
max |difference|                = 1.24e-14
identical predictions           : True
support vectors: kernel 38, explicit 38
```

Not merely the same predictions — the same *decision function values*, to fourteen decimal places, and the same 38 support vectors. They are one model, computed two ways.

The RBF kernel, where there is no map to build

$$K(x, z) = \exp\left(-\gamma \|x - z\|^2\right)$$

```
gamma =   0.1   exp(-g||u-v||^2) = 0.8994246481   sklearn = 0.8994246481
gamma =     1   exp(-g||u-v||^2) = 0.3464558103   sklearn = 0.3464558103
gamma =    10   exp(-g||u-v||^2) = 0.0000249160   sklearn = 0.0000249160
||u-v||^2 = 1.0600
```

Expand the exponential

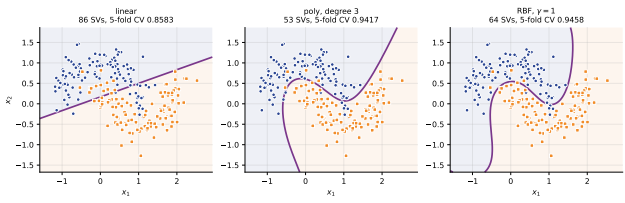
e^t has infinitely many terms, so the implied feature map has infinitely many coordinates. You could never write it down.

And you never need to

The kernel is one subtraction, one norm and one exp. An SVM can therefore fit a linear model in an infinite-dimensional space using arithmetic that fits on this slide.

Three kernels on the same two moons

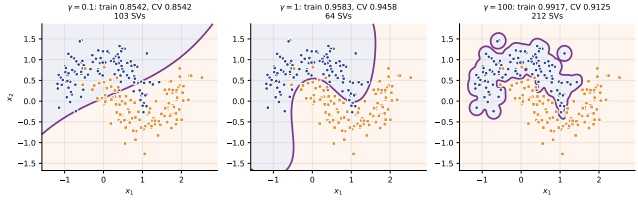
kernel	nSV	train	5-fold CV
linear	86	0.8667	0.8583
poly deg 2	84	0.8542	0.8417
poly deg 3	53	0.9458	0.9417
rbf g=1	64	0.9583	0.9458



Degree *two* is worse than linear here (0.8417 against 0.8583). An even-degree polynomial is symmetric in a way this data is not. More flexibility is not automatically better — it has to be the *right* flexibility.

γ is how far one training point reaches

gamma	distance at which K falls to 0.01	nSV	train	5-fold CV
0.01	21.4597	158	0.8208	0.8167
0.1	6.7861	103	0.8542	0.8542
1	2.1460	64	0.9583	0.9458
10	0.6786	89	0.9708	0.9625
100	0.2146	212	0.9917	0.9125

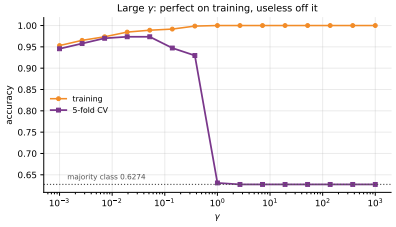


Small γ: every point influences the whole plane, and the boundary is nearly straight. Large γ: influence dies within 0.21 units, so the model builds an island around each training point.

Large γ overfits, and it is spectacular

gamma	train	5-fold CV	nSV of 569
0.001	0.9525	0.9455	200
0.01	0.9824	0.9701	111
0.1	0.9912	0.9596	221
1	1.0000	0.6309	568
10	1.0000	0.6274	569

majority class = 0.6274



At $\gamma = 10$: training **1.0000**, cross-validated **0.6274** — *exactly* the majority-class rate, and **569 of 569** rows are support vectors. The model has memorised every row and generalises to nothing.

Predict first #4 — what happens if you skip the scaler?

Commit to a number

`breast_cancer` has 30 columns. The widest has standard deviation 568.86; the narrowest, 0.002644. That is a ratio of **215 171**.

How much cross-validated accuracy does an RBF SVM lose if you fit the raw columns instead of standardising them first?

Predict first #4 — what happens if you skip the scaler?

Commit to a number

`breast_cancer` has 30 columns. The widest has standard deviation 568.86; the narrowest, 0.002644. That is a ratio of **215 171**.

How much cross-validated accuracy does an RBF SVM lose if you fit the raw columns instead of standardising them first?

0.9210 raw against **0.9771** scaled — a loss of **0.0561**.

Fifty-six patients in a thousand, given away by one missing line of code.

Scaling is not optional — with one honest exception

```
breast_cancer: widest column sd 568.8565, narrowest 0.002644, ratio 215170.7
model                          raw   scaled      gap
SVC(rbf, gamma='scale')      0.9210   0.9771  +0.0561
SVC(rbf, gamma=1)            0.6274   0.6309  +0.0035
SVC(linear)                  0.9526   0.9754  +0.0229
SVC(poly, degree 3)          0.9122   0.8981  -0.0142
```

Why it bites so hard

$\|x - z\|^2$ is dominated by whichever column has the largest numbers. One column with an $s.d.$ of 569 drowns out twenty-nine others, so the RBF kernel effectively sees a one-feature problem.

Report the row that disagrees

The degree-3 polynomial got **worse** when scaled, -0.0142. It happens; do not hide it. Scaling is a reliable default, not a theorem — and both of its poly scores are below the scaled RBF anyway.

Always inside a `Pipeline`, never as a separate step on the whole table: fitting the scaler on the test rows is leakage.

The whole recipe, in six lines

```
pipe = Pipeline([("scale", StandardScaler()), ("svm", SVC(kernel="rbf"))])
grid = {"svm__C": [0.1, 1, 10, 100], "svm__gamma": [0.001, 0.01, 0.1, 1]}
gs = GridSearchCV(pipe, grid, cv=StratifiedKFold(5, shuffle=True,
                  random_state=0)).fit(Xtr, ytr)
```

```
best parameters : {'svm__C': 100, 'svm__gamma': 0.001}
best CV score   : 0.9859
held-out score  : 0.9580
support vectors : 43 of 426 training rows
```

Note the held-out score is *below* the cross-validated one: 0.9580 against 0.9859. That gap is the optimism of having chosen the best of sixteen configurations. **Report the held-out number.** And note C and γ traded off — a large C with a tiny γ.

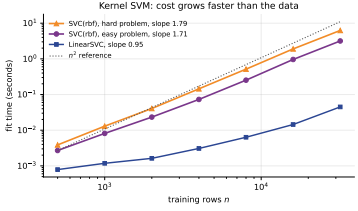
What it costs as the data grows — count first

```
     n     nSV easy   nSV hard   kernel entries n^2
   500         248        380              250000
  2000         624       1097             4000000
  8000        1604       3108            64000000
 32000        4596       9280          1024000000
nSV grows as n^0.69 (class_sep 0.9) and n^0.76 (class_sep 0.4)
64x the rows gives 18.5x the support vectors on the easy problem and
24.4x on the hard one; the kernel matrix grows 4096x either way
```

These are exact, and the same on your machine as on mine. The kernel matrix grows as n^2; the support-vector set grows as $n^{0.69}$. The cost lives between the two, which is why the measured exponent lands between 1 and 2.

And now the clock, with a health warning

```
     n   SVC(rbf) fit   nSV / n   LinearSVC fit    ratio
   500         0.003 s      0.50         0.001 s      3.6x
  2000         0.024 s      0.31         0.002 s     13.6x
  8000         0.257 s      0.20         0.005 s     47.1x
 32000         3.308 s      0.14         0.048 s     68.7x
MACHINE-DEPENDENT: time ~ n^1.72 (easy), n^1.81 (hard), n^0.98 (LinearSVC)
```



A seed cannot fix a clock. Repeated runs on this machine moved the exponent between $n^{1.68}$ and $n^{1.72}$. Quote the counts.

Be precise about the exponent

What the textbook says

Kernel SVM training is between $O(n^2)$ and $O(n^3)$, because the kernel matrix alone has n^2 entries.

What we measured

An exponent *below* quadratic on both problems — so say so, and then explain it.

And the reason is in the table

The support-vector fraction **falls** as n grows: $0.50 \to 0.14$. `libsvm` caches and shrinks, and the work is driven by the support vectors, not the rows. Make the problem harder — keep more support vectors — and the *count* exponent climbs from 0.69 to 0.76, and the time follows it.

The practical conclusion survives intact, stated in counts: $64\times$ the rows gives **18.5**$\times$ the support vectors and **4096**$\times$ the kernel matrix. Beyond roughly 10^5 rows, use `LinearSVC` — which never forms that matrix — or `SGDClassifier`.

Prediction is not free either — and there are only two sic

```
C =   0.01  nSV =  5988  kernel evaluations to score 8000 rows = 47904000
C =      1  nSV =  1604  kernel evaluations to score 8000 rows = 12832000
C =    100  nSV =  1368  kernel evaluations to score 8000 rows = 10944000
```

Why

$$f(x) = \sum_{i \in \mathrm{SV}} \alpha_i y_i K(x_i, x) + b.$$

Every prediction is a sum over the support vectors, so scoring 8000 rows costs exactly $8000 n_{\mathrm{SV}}$ kernel evaluations — $4.4\times$ more at $C = 0.01$ than at $C = 100$, on any machine. An RBF SVM is a *learned*, weighted, sparse nearest-neighbour rule — and it inherits k-NN's prediction-time cost.

```
classes             : 3
pairwise SVMs       : 3  (= k(k-1)/2)
dual_coef_ shape    : (2, 51)
n_support_ per class: [ 8 22 21]
decision_function   : (1, 3) one per pair
5-fold CV accuracy : 0.9533
```

An SVM is intrinsically a **two-class** machine: one hyperplane, two sides. `SVC` handles k classes with $k(k-1)/2$ one-against-one machines and a vote. At $k = 10$ that is 45 models.

Summary — twelve lines

1. Many hyperplanes separate a separable set and accuracy cannot rank them. Of five perfect lines on the six points, clearance ranged from 0.0707 to **0.7071**.
2. The signed distance to $w^\top x + b = 0$ is $(w^\top x + b)/\|w\|$. Fix the scale so the closest points have $y_i f(x_i) = 1$, and the band has width $\mathbf{2}/\|\mathbf{w}\|$. Maximising it is minimising $\frac{1}{2}\|w\|^2$ — a convex QP with one global optimum.
3. The Lagrangian gives $w = \sum_i \alpha_i y_i x_i$ and $\sum_i \alpha_i y_i = 0$; substituting yields the dual, in which the data appears **only** as $\langle x_i, x_j \rangle$.
4. Complementary slackness $\alpha_i[y_i f(x_i) - 1] = 0$ forces every point to have either $\alpha_i = 0$ or $y_i f(x_i) = 1$. The rest are **support vectors**.
5. Six points by hand: $\alpha_A = \alpha_D = 1$, $w = (1, 1)$, $b = -3$, width **1.414214**. `SVC(kernel='linear', C=1e6)` returns the identical numbers to 10^{-9}.
6. Deleting non-support-vector B moved w by **0.00e+00**. Deleting support vector A moved w to $(0.6664, 0.6668)$ and widened the margin to 2.1215. Refitting on the 11 support vectors of an 80-point problem gave identical predictions on all 80.
7. Soft margin: $\xi_i \geq 0$, objective $\frac{1}{2}\|w\|^2 + C\sum \xi_i$, dual unchanged except $0 \leq \alpha_i \leq C$. Across $C = 0.01 \to 1000$ the width went $5.2135 \to 1.3483$ and points inside the margin $32 \to 5$, while training accuracy stayed at 0.9625 throughout.
8. C is **inverse** regularisation: $\|w\|$ climbed $0.3555 \to 20.8998$ as C went $0.001 \to 100$, and CV peaked at **0.9772** in the middle.
9. Eliminating ξ gives $\sum \max(0, 1 - y_i f(x_i)) + \frac{1}{2C}\|w\|^2$: the SVM is the model whose loss is the **hinge**. It is flat beyond $m = 1$, which is exactly why most α_i are zero — 11 of 80 points had $m \leq 1$ and `n_support_` was 11.
10. Kernels: $\langle \varphi(x), \varphi(z) \rangle = (1 + x^\top z)^2$ **verified** to 0.00e+00 at $x = (1, 2)$, $z = (3, 1)$ (both sides 36), and to 2.8×10^{-14} over a 200×200 Gram matrix. Kernel machine and explicit-map machine agree to 1.24×10^{-14} with the same 38 support vectors. Circles: $0.5682 \to$ **1.0000**.
11. γ is a reach. On `breast_cancer`, $\gamma = 10$ gives train 1.0000, CV **0.6274** (the majority-class rate) and 569/569 support vectors. Without `StandardScaler` the RBF SVM loses **0.0561** — the column standard deviations differ by a factor of 215 171.
12. Cost: 64× the rows gives 18.5× the support vectors and 4096× the kernel matrix — exact. The measured time exponent lands below the documented $O(n^2)$–$O(n^3)$ for that reason. Prediction cost scales with the number of support vectors. Past $\sim 10^5$ rows, use `LinearSVC`.

Before the next lecture

Read

`notes.pdf` — the $2/\|w\|$ derivation in full, the Lagrangian and the dual step by step, the six points solved twice, and **ten practice questions with full worked solutions**.

Do

Reproduce the six-point solution with a pen: guess the support set, use $\sum \alpha_i y_i = 0$ and the two margin equations, get $\alpha = 1$, $w = (1, 1)$, $b = -3$. Then verify $(1 + x^\top z)^2 = \langle \varphi(x), \varphi(z) \rangle$ for $x = (1, 2)$, $z = (3, 1)$ by expanding both sides.

The habit to take away

`make_pipeline(StandardScaler(), SVC())` and a grid over C and γ. Never an SVC on raw columns; never C or γ chosen on the training score.

And the habit to unlearn

"Large C regularises more." It is the opposite: $\lambda = 1/C$. If you remember one sentence from the soft-margin section, make it that one.

Materials: `https://github.com/tali7c/Applied-Machine-Learning`

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 8–9.;James et al., *An Introduction to Statistical Learning with Python*, Ch. 8–9.;Bishop, *Pattern Recognition and Machine Learning*, Ch. 4–5, 7, 14.;Zhang et al., *Dive into Deep Learning*, Ch. 5.